FIAP

Enterprise Challenge 2026 • Sociedade 5.0

SMART HAS — DIGITAL 360

Atividade Consolidando o Smart HAS

Consolidação em Oracle PL/SQL e pipeline de entrega contínua

Nome completo	RM
Eduardo Andrade Martins Vasques	556970
Otávio Ramos dos Santos Souza	550361
Enzo Miranda Ward de Paiva	557632
Rafael Pinto de Albuquerque	559136
Guilherme Leoni Vidigal Tibúrcio	557500

São Paulo • Novembro de 2026

Sumário

Resumo executivo.....
1. Arquitetura da solução.....
2. Parte 1 — Consolidação em Oracle PL/SQL.....
   2.1 Packages.....
   2.2 Subprogramas de cada package.....
   2.3 Dynamic SQL.....
   2.4 Triggers.....
   2.5 Demais objetos.....
   2.6 Integração com o back-end.....
3. Parte 2 — Automação e entrega contínua.....
   3.1 Pipeline.....
   3.2 Deploy com integração ao Oracle.....
   3.3 Logs e relatórios.....
4. Validação e evidências.....
5. Como executar.....
6. Considerações finais e links.....

Resumo executivo

Esta entrega leva o **Smart HAS / Digital 360** à maturidade técnica em duas frentes. Na camada de dados, as regras de negócio que estavam em functions e procedures soltas foram consolidadas em **4 packages PL/SQL**, com **2 triggers** e **Dynamic SQL** para consultas em tempo real. Na entrega, um **pipeline de CI/CD no GitHub Actions** passou a testar todas as camadas, publicar a imagem do back-end e implantá-la junto de um banco Oracle real, gerando logs e relatórios a cada execução.

Pedido no enunciado	Como foi atendido
Agrupar em Packages	`PKG_SH_RISCO`, `PKG_SH_IOT`, `PKG_SH_USUARIO` e `PKG_SH_CONSULTA`
Trigger de auditoria ou automação	`TRG_SH_ALERTA_AUDITORIA` (auditoria) e `TRG_SH_PEDIDO_ENTREGUE` (automação)
Dynamic SQL	`FN_BUSCAR_PEDIDOS` (`DBMS_SQL`) e `FN_RESUMO_POR_DIMENSAO` (`OPEN FOR`)
Documentar os objetos	Seção 2 deste documento e comentários nos scripts
Build e testes automatizados	Jobs Flutter, Python, Angular, Java e Oracle PL/SQL
Deploy do back-end + Oracle	Imagem no GHCR e job de deploy em homologação com smoke test
Logs e relatórios	Resumo em cada execução e artefatos `relatorio-*`

1. Arquitetura da solução

O aplicativo Flutter e o dashboard Angular consomem o back-end Spring Boot. O back-end usa H2 para os dados da aplicação e aciona a camada Oracle por JDBC, nas rotas `/oracle/**`. As regras de risco, alertas e relatórios ficam no banco, em um só lugar.

```
App Flutter / Dashboard Angular
        |  REST + JWT
Back-end Spring Boot (pacote plsql)
        |  JDBC (CallableStatement, REF CURSOR)
Oracle: packages -> view -> tabelas <- triggers -> auditoria
```

2. Parte 1 — Consolidação em Oracle PL/SQL

2.1 Packages

Cada package reúne as rotinas de um assunto. A especificação expõe só o que os clientes podem chamar, e o corpo guarda a implementação e os auxiliares privados. O valor prático: a API passou a depender de um contrato estável, e nenhum subprograma ficou solto no schema (um teste automatizado garante isso).

Package	Finalidade	Valor agregado
`PKG_SH_RISCO`	Motor de risco da AI Logistics: score, nível, resumo e recálculo	Uma única regra de risco para API, relatórios e rotinas, com histórico de cada decisão
`PKG_SH_IOT`	Monitoramento dos sensores e ciclo de vida dos alertas	Detecta automaticamente leituras fora da faixa segura e registra o encerramento dos alertas
`PKG_SH_USUARIO`	Engajamento dos usuários nas trilhas de aprendizagem	Identifica quem precisa de tutoria, com snapshots que permitem comparar a evolução

Package	Finalidade	Valor agregado
PKG_SH_CONSULTA	Consultas flexíveis em tempo real com Dynamic SQL	O painel combina filtros, ordem e agrupamento sem exigir uma consulta para cada combinação

2.2 Subprogramas de cada package

PKG_SH_RISCO

Subprograma	Tipo	Finalidade e valor agregado
FN_NIVEL_RISCO	Function	Converte o score em BAIXO, MEDIO, ALTO ou CRITICO. É determinística, então pode ser usada em SELECT e WHERE.
FN_CALCULA_RISCO	Function	Calcula o risco de atraso (0 a 100) por histórico, reagendamentos, estoque, distância e status. É o indicador central da logística.
FN_RESUMO_PEDIDO	Function	Devolve uma linha pronta para exibição em app, painel ou notificação.
SP_RECALCULAR_RISCO	Procedure	Recalcula, grava o histórico e abre alerta se ALTO ou CRITICO, sem duplicar e sem reabrir pedido entregue. Acionada pela API.

PKG_SH_IOT

Subprograma	Tipo	Finalidade e valor agregado
SP_REGISTRAR_ALERTAS_SENSORES	Procedure	Percorre as leituras pendentes com cursor FOR UPDATE e gera os alertas. Pode ser reexecutada sem duplicar.
SP_RESOLVER_ALERTA	Procedure	Encerra um alerta. A mudança fica auditada pela trigger. **Novo nesta entrega.**

PKG_SH_USUARIO

Subprograma	Tipo	Finalidade e valor agregado
FN_PROGRESSO_MEDIO_USUARIO	Function	Média de progresso do usuário nas trilhas.
SP_GERAR_RELATORIO_USUARIOS	Procedure	Gera o snapshot por usuário e o classifica em ENGAJADO, EM_PROGRESSO ou INICIANTE.

PKG_SH_CONSULTA

Subprograma	Tipo	Finalidade e valor agregado
FN_BUSCAR_PEDIDOS	Function	Busca com filtros e ordem opcionais (Dynamic SQL). **Novo nesta entrega.**
FN_RESUMO_POR_DIMENSAO	Function	Indicadores agrupados por status, nível, região ou parceiro (Dynamic SQL). **Novo nesta entrega.**

2.3 Dynamic SQL

Por que é necessário: o operador do painel decide em tempo real quais filtros combinar e como ordenar. Com SQL estático, seria preciso uma consulta por combinação ou condições como `(:p IS NULL OR coluna = :p)`, que atrapalham o otimizador. Aqui, só os filtros informados entram no comando.

O FN_BUSCAR_PEDIDOS usa DBMS_SQL porque a quantidade de variáveis de ligação muda a cada chamada, e devolve um SYS_REFCURSOR que o Java lê normalmente:

```
IF v_status IS NOT NULL THEN
    v_sql := v_sql || ' AND ST_PEDIDO = :status';
END IF;
...
v_cursor := DBMS_SQL.OPEN_CURSOR;
DBMS_SQL.PARSE(v_cursor, v_sql, DBMS_SQL.NATIVE);
IF v_status IS NOT NULL THEN
    DBMS_SQL.BIND_VARIABLE(v_cursor, ':status', v_status);
END IF;
v_linhas := DBMS_SQL.EXECUTE(v_cursor);
RETURN DBMS_SQL.TO_REFCURSOR(v_cursor);
```

O `FN_RESUMO_POR_DIMENSAO` usa SQL dinâmico nativo (`OPEN FOR`), porque só a coluna de agrupamento muda.

Proteção contra SQL Injection: valores do usuário entram sempre como variável de ligação. Nomes de coluna nunca vêm do cliente: ele informa um apelido (`score`, `prazo`, `parceiro`) que o package traduz por uma lista fechada. Qualquer outro valor gera o erro `-20002`, devolvido pela API como HTTP 400. Os testes automatizados cobrem tentativas de injeção no filtro, na ordenação e na direção.

2.4 Triggers

Trigger	Tipo	Finalidade	Valor agregado
`TRG_SH_ALERTA_AUDITORIA`	Auditoria	A cada INSERT, DELETE ou mudança de status em `T_SH_ALERTA`, grava quem, quando, de onde e o que mudou	Rastreabilidade para qualquer cliente (API ou SQL Developer), sem depender de a aplicação lembrar de registrar
`TRG_SH_PEDIDO_ENTREGUE`	Automação	Quando o pedido passa para ENTREGUE, encerra seus alertas abertos	O operador só vê o que ainda exige ação, sem trabalho manual

As duas se complementam: o encerramento automático dispara a trigger de auditoria, então também fica registrado. A auditoria participa da transação do alerta: se a alteração sofrer ROLLBACK, o registro é desfeito junto.

2.5 Demais objetos

Objeto	Finalidade
`T_SH_ALERTA_AUDITORIA`	Trilha de auditoria. Sem chave estrangeira de propósito: o registro precisa sobreviver à exclusão do alerta. **Nova nesta entrega.**
`SEQ_SH_AUDITORIA`	Numeração da auditoria. **Nova nesta entrega.**
`V_SH_PEDIDO_RISCO`	View com o risco calculado por pedido, usada pela API e pelo `PKG_SH_CONSULTA`
11 tabelas da Fase 6	Usuários, cursos, matrículas, serviços, favoritos, pedidos, sensores, leituras, alertas, histórico de risco e relatórios (modelo documentado na entrega anterior)

2.6 Integração com o back-end

Endpoint	Objeto Oracle
`GET /oracle/pedidos/busca`	`PKG_SH_CONSULTA.FN_BUSCAR_PEDIDOS`
`GET /oracle/pedidos/resumo?dimensao=`	`PKG_SH_CONSULTA.FN_RESUMO_POR_DIMENSAO`
`POST /oracle/alertas/{id}/resolver` (ADMIN)	`PKG_SH_IOT.SP_RESOLVER_ALERTA`

Endpoint	Objeto Oracle
`GET /oracle/alertas/auditoria`	`T_SH_ALERTA_AUDITORIA`
`POST /oracle/entregas/{id}/recalcular-risco`	`PKG_SH_RISCO.SP_RECALCULAR_RISCO`
`POST /oracle/sensores/{id}/leituras`	`PKG_SH_IOT.SP_REGISTRAR_ALERTAS_SENSORES`
`POST /oracle/relatorios/usuarios`	`PKG_SH_USUARIO.SP_GERAR_RELATORIO_USUARIOS`

3. Parte 2 — Automação e entrega contínua

3.1 Pipeline

O pipeline está em `.github/workflows/ci-cd.yml` e roda a cada push em `main` ou em branches `fase*`, em pull requests e sob demanda. Os testes vêm primeiro: a imagem só é publicada se todas as camadas passarem.

```
Flutter ----------+
Python  ----------+
Angular ----------+--> Publicar imagem --> Deploy em    --> Relatorio
Java    ----------+    no GHCR            homologacao      final
Oracle PL/SQL ----+
```

Job	O que faz
`flutter`	Análise estática, testes e build web do aplicativo
`backend-python`	Testes com pytest da API FastAPI
`dashboard`	Instalação das dependências e build do Angular
`backend-java`	Build, testes JUnit e cobertura com JaCoCo
`oracle-plsql`	Sobe um Oracle real, instala os objetos e roda os testes PL/SQL
`publicar-imagem`	Constrói a imagem do back-end e publica no GitHub Container Registry, marcada com o commit
`deploy-homologacao`	Baixa a imagem publicada, sobe o Oracle, instala a camada PL/SQL, sobe o back-end e roda o smoke test
`relatorio`	Consolida o resultado de todas as etapas e falha se alguma tiver falhado

3.2 Deploy com integração ao Oracle

O deploy usa um **ambiente de homologação temporário**, criado no próprio runner com `deploy/docker-compose.homolog.yml`: o Oracle é instalado do zero por `database-oracle/00_instalar.sql` e o back-end vem da imagem publicada. O ambiente é encerrado no fim da execução, portanto não fica uma URL pública no ar. A imagem publicada no GHCR é o que seria levado a um servidor definitivo. O mesmo script, `deploy/deploy_homolog.sh`, roda na máquina de qualquer integrante do grupo.

O SQL*Plus roda com `WHENEVER SQLERROR EXIT FAILURE`, e o script `98_verificar_objetos.sql` transforma erro de compilação em falha. Assim, um banco quebrado interrompe o deploy. As credenciais do ambiente são geradas a cada execução e mascaradas nos logs.

O smoke test (`deploy/smoke_test.sh`) percorre o caminho completo REST → Java → JDBC → Oracle: autenticação e autorização, os quatro packages, as tentativas de SQL Injection, a gravação da auditoria pela trigger e a continuidade do H2 da Fase 5.

3.3 Logs e relatórios

Artefato	Conteúdo
Resumo da execução	Tabela com o resultado de cada etapa, testes JUnit, cobertura, resultado PL/SQL, imagem publicada e smoke test
`relatorio-java`	Relatórios do Surefire e cobertura JaCoCo em HTML
`relatorio-oracle`	Log da instalação e resultado de cada teste PL/SQL
`relatorio-deploy`	Resultado do smoke test, logs da API e do Oracle, estado dos containers e manifesto (imagem, commit e data)
`relatorio-python` e `relatorio-flutter`	Resultados dos testes dessas camadas

4. Validação e evidências

Tudo abaixo foi executado de verdade, com Oracle Database Free em Docker. Os arquivos estão em `docs/fase7/evidencias/`.

Verificação	Resultado	Evidência
Testes PL/SQL automatizados	47 OK, 0 falhas	`01_testes_plsql.txt`
Testes JUnit do back-end	17 OK, 0 falhas	Artefato `relatorio-java`
Smoke test contra o Oracle	20 OK, 0 falhas	`03_smoke_test_homologacao.md`
Todos os objetos com status VALID	Confirmado	`98_verificar_objetos.sql`
Package com erro de compilação interrompe o script	Confirmado (saída 1)	Teste negativo manual
Teste PL/SQL com resultado errado interrompe o script	Confirmado (saída 1)	Teste negativo manual
Pipeline completo no GitHub Actions	Em validação	

Correções encontradas no caminho: um pedido já entregue podia reabrir alerta ao ter o risco recalculado, e um parâmetro de consulta ausente ou inválido devolvia HTTP 500 em vez de 400. Ambas foram corrigidas e têm verificação automatizada.

5. Como executar

Requisito: Docker. Na raiz do repositório:

```
bash deploy/deploy_homolog.sh
# API:     http://localhost:8080/swagger-ui.html  (admin / admin123)
# Oracle:  localhost:1521/FREEPDB1  (smarthas / smarthas)

docker compose -f deploy/docker-compose.homolog.yml down -v   # encerra
```

Para ver os testes e a demonstração completa no banco, execute `@00_executar_tudo.sql` a partir da pasta `database-oracle`.

6. Considerações finais e links

A solução saiu desta etapa mais organizada, rastreável e segura de evoluir: as regras do banco estão agrupadas em contratos claros, as mudanças em alertas deixam rastro e nenhuma versão chega à homologação sem passar pelos testes de todas as camadas.

Repositório: https://github.com/DevRafonalde/digital360_flutter

Vídeo (YouTube, não listado): [INSERIR LINK DO VÍDEO]